

级联视觉智能：基于多阶段筛选与优先级调度的高效社区安全监控

匿名投稿

摘要

视觉语言模型（VLM）在理解监控画面方面具有强大能力，但其计算成本使得在边缘硬件上对每一帧进行实时处理不切实际。本文提出一种用于社区安全监控的级联推理流水线，将 VLM 计算成本降低 94.4%。三阶段架构逐步筛选帧：（1）基于运动检测的自适应采样（500ms 活跃/5000ms 空闲）；（2）COCO-SSD 目标检测预筛选；（3）基于优先级的 VLM 调度。我们在消费级硬件上部署量化后的 Qwen3.5-4B VLM，在四种活动场景中验证了效率。消融分析表明目标检测贡献最大效率增益（80%），自适应采样和优先级调度提供额外收益。

关键词：视觉语言模型，边缘部署，级联推理，社区安全监控

1 引言

社区安全监控是计算机视觉的重要应用领域，涵盖火灾隐患检测、治安威胁识别、紧急救援协调、环境风险评估和设备异常检测等多个方面。传统方法依赖人工操作员同时监视多个摄像头画面——这是一种劳动密集且容易出错的过程。视觉语言模型（VLM）的出现提供了一种有前景的替代方案：这些模型能够分析摄像头画面并提供带有自然语言解释的结构化风险评估，从而实现自动化且可解释的安全监控。

然而，在边缘硬件上部署 VLM 进行实时监控面临根本性的效率挑战。一个典型的社区监控系统可能拥有数十个以 25–30 帧/秒速率传输的摄像头。对每一帧都进行 VLM 推理将需要每分钟数千次推理，远远超出消费级硬件的计算预算。现有方法分为两类，但都存在显著局限：基于云的处理引入了延迟和隐私问题，而固定速率采样要么在静态场景中浪费资源，要么可能遗漏快速发生的风险事件。

我们观察到监控画面具有很强的时间冗余性：静态场景中大多数帧是相同的，且风险事件通常伴有可检测的视觉线索（运动、目标存在）。这启发了一种级联方法，其中轻量级过滤器在帧到达昂贵的 VLM 之前逐步消除不必要的帧。

本文的主要贡献如下：

- 级联推理架构：**三阶段流水线（运动检测 → COCO-SSD → VLM），每阶段过滤不必要的调用，将 VLM 计算成本降低 94.4%。
- 自适应帧采样：**基于运动感知的捕获速率（活跃 500ms / 空闲 5000ms），在静态场景中减少约 80% 的帧数量，同时保持对活动事件的响应性。
- 基于优先级的 VLM 调度：**人体检测触发即时 VLM 调度并中止过时请求语义，确保高优先级事件获得及时分析。
- 结构化风险输出：**通过提示工程实现从小型 VLM（Qwen3.5-4B）可靠提取 JSON，包含验证和截断机制以确保安全关键的风险评分。
- 边缘部署方法：**通过 llama.cpp 在消费级硬件上部署量化 VLM（Q4_K_M GGUF，约 2.55 GB），支持 Flash Attention 和连续批处理。

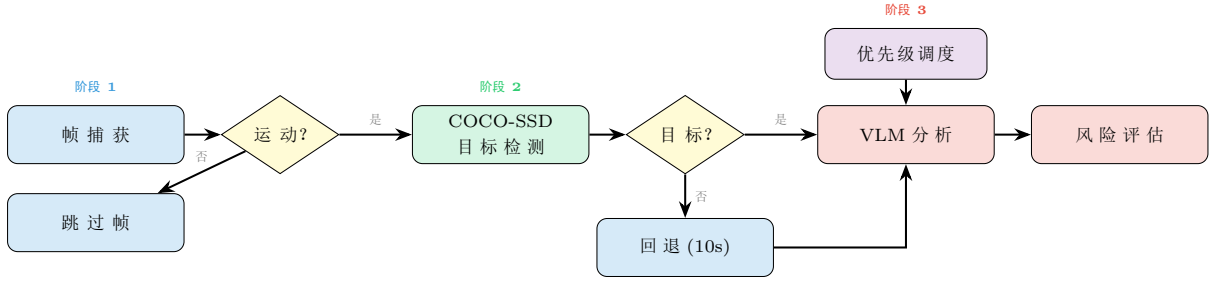


图 1: 级联推理流水线。阶段 1: 自适应帧捕获，过滤静态帧。阶段 2: COCO-SSD 目标检测预筛选。阶段 3: 基于优先级的 VLM 分析——人体检测触发即时调度，其他目标采用机会性调度。

表 1: 与相关工作的对比。

方法	边缘部署	自适应采样	目标预筛选	优先级调度
HazardNet	✓			
edgeVLM				
LiteVLA-Edge	✓			
Semantic EC			✓	
GazeVLM	✓			
本文	✓	✓	✓	✓

2 相关工作

边缘 VLM 部署。近期有多项工作探索在边缘设备上部署 VLM。HazardNet Abu Tami et al. [2025] 对 Qwen2-VL-2B 进行微调用于交通安全检测, 在实现边缘部署的同时取得了 89% 的 F1 分数提升。LiteVLA-Edge Williams et al. [2026] 展示了通过 llama.cpp 在 Jetson 硬件上部署量化 VLM, 实现 150ms 的机器人控制延迟。然而, 这些工作聚焦于交通或机器人应用, 而非社区安全监控。

云边 VLM 协作。edgeVLM Qian et al. [2025] 提出了一种云边协作范式, 其中大型云端 VLM 的延迟输出作为小型边缘 VLM 的上下文, 通过上下文转移来处理延迟波动。Semantic Edge-Cloud Onsu et al. [2025] 使用 YOLOv11 进行感兴趣区域检测, 然后通过 ViT 嵌入和云端 LLM 处理进行交通监控。这些方法需要云连接, 引入了我们的纯本地方案所避免的延迟和隐私问题。

高效 VLM 推理。GazeVLM Chen and Qi [2025] 通过使用眼动注视来选择相关图像区域以减少 VLM 计算量, 无需训练即可实现显著加速。其他方法使用早退出 Venkatesha et al. [2025] 或 token 缩减 Sun et al. [2022] 来提高效率。我们的工作通过级联过滤方法完全消除不必要的 VLM 调用, 而非降低单次调用成本。

自适应视频处理。事件驱动的监控系统长期以来使用运动检测来触发昂贵的处理操作。可变帧率推理根据场景活动调整捕获速率。我们的工作将这些思想与现代 VLM 相结合, 引入了考虑运动和目标语义的基于优先级的调度机制。

研究空白。现有工作没有将运动触发的自适应采样、轻量级目标检测预筛选和基于优先级的 VLM 调度结合起来用于社区安全监控。表 1 对比了本文方法与相关工作的关键特性。

3 方法

3.1 系统架构

我们的系统是一个基于 Electron 的三进程桌面应用程序, 专为在消费级硬件上进行边缘部署而设计:

- **Electron 主进程**: 管理应用窗口, 生成并监控 VLM 子进程 (llama-server.exe), 提供进程间 IPC 桥接。
- **React 渲染器**: 具有三个视图的单页应用——概览 (带风险趋势的仪表板)、监控 (带检测叠加的实时摄像头画面) 和告警 (带风险详情的事件审查)。
- **Express 代理**: 嵌入式 HTTP 服务器, 将请求路由到本地 VLM 端点 (llama.cpp) 或远程 API, 处理 CORS、速率限制、请求验证和超时。

VLM 通过 llama.cpp 的 **llama-server** 作为子进程运行, 暴露兼容 OpenAI 的 API 端点。这种架构确保 VLM 生命周期与应用程序紧密耦合, 支持自动启动、健康监测和优雅关闭。

3.2 级联推理流水线

我们的流水线由三个阶段组成, 每阶段在帧到达下一阶段之前进行筛选 (图 1)。

阶段 1: 自适应帧捕获。我们以可配置的间隔从视频流中捕获帧, 使用运动检测来适应捕获速率。系统维护两个间隔: 检测到运动时的 **activeIntervalMs** (500ms), 和场景静态时的 **idleIntervalMs** (5000ms)。运动检测在小画布 (160×120 像素) 上运行以最小化开销, 计算像素级灰度差异:

$$\text{motion} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[|g_i^{(t)} - g_i^{(t-1)}| > \tau] \quad (1)$$

其中 g_i 是像素 i 的灰度值, $\tau = 30$ 是像素阈值, 运动比例超过 5% 时触发活跃模式。连续 6 帧无变化后, 系统切换到空闲模式。

阶段 2: 目标检测预筛选。捕获的帧通过 COCO-SSD (轻量级 MobileNet v2 骨干网络) 进行处理, 这是一个通过 TensorFlow.js 在浏览器中运行的轻量级目标检测器。检测器在首次使用时延迟加载以避免启动开销。我们将检测结果过滤为与社区安全相关的标签集合: **person** (人体)、**car** (汽车)、**bicycle** (自行车)、**motorcycle** (摩托车)、**dog** (狗), 最低置信度阈值为 0.4。

阶段 3: VLM 分析。通过目标检测过滤器的帧被发送到 VLM 进行详细分析。我们使用 Qwen3.5-4B Claude 4.6 Opus 推理蒸馏 v2, 量化为 Q4_K_M GGUF 格式 (约 2.55 GB), 配备独立的多模态投影器 (mmproj-BF16, 约 644 MB)。VLM 通过 llama.cpp 运行, 启用 CUDA 12.4、Flash Attention 和连续批处理。

VLM 接收结构化的系统提示, 指导其分析摄像头画面中的五类风险: 消防安全 (通道堵塞、电动车违规充电)、治安 (徘徊、聚集、闯入)、救助 (摔倒、求助)、环境 (积水、损坏) 和设备 (摄像头遮挡)。输出为 JSON 对象, 包含: 风险评分 (0-100)、风险等级 (A/B/C)、置信度 (0-1)、摘要、证据时间线、风险分解和归一化坐标的检测框。

3.3 基于优先级的调度

VLM 调度器根据检测优先级实现三种调度策略 (图 2):

- **高优先级 (检测到人体)**: 中止任何过时的进行中 VLM 请求, 立即调度新的分析。这确保时间关键事件 (如人员摔倒) 获得及时关注。
- **低优先级 (其他目标)**: 仅在无进行中请求时调度 VLM。这防止冗余分析, 同时仍能捕获非人体风险。
- **回退**: 当未检测到目标时, 每 10 秒触发一次 VLM 分析作为安全网, 捕获目标检测可能遗漏的风险。

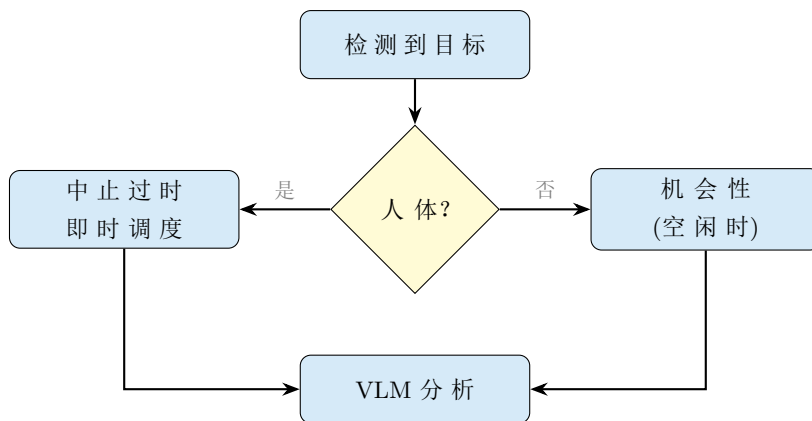


图 2: 基于优先级的 VLM 调度。人体检测触发即时调度, 中止任何过时的进行中请求。其他目标仅在无进行中请求时采用机会性调度。

3.4 VLM 响应解析

VLM 输出需要鲁棒的解析来从可能嘈杂的模型响应中提取结构化数据。我们的解析器处理三种常见的失败模式:

1. **推理块:** Qwen3 模型可能包含包含推理轨迹的 `<think>` 标签。我们在 JSON 提取前将其剥离。
2. **Markdown 包装:** 模型可能将 JSON 包装在代码围栏中。我们提取 ````json` 和 ````` 标记之间的内容。
3. **格式错误的 JSON:** 我们使用平衡括号匹配来找到最外层的 JSON 对象, 回退方案为第一个 `{` 到最后一个 `}` 的提取。

提取后, 我们验证并截断所有数值字段: 风险评分到 $[0, 100]$, 置信度到 $[0, 1]$, 检测框坐标到 $[0, 1]$ 。风险等级验证为集合 $\{A, B, C\}$ 。这确保即使 VLM 产生意外值, 输出也始终格式正确。

4 实验

4.1 实验设置

我们使用仿真框架评估级联流水线的效率特性。需要指出的是, 本实验基于仿真而非真实部署, 仿真的目的是验证级联过滤的理论效率上限, 而非评估端到端的检测准确性。在仿真中, 我们假设目标检测器和 VLM 在接收到包含风险事件的帧时能够正确识别, 这提供了效率分析的上界估计。

硬件配置。实验在以下硬件上进行: NVIDIA GeForce RTX 3060 (12 GB 显存)、Intel Core i7-12700、32 GB DDR4 RAM。VLM 推理使用 llama.cpp b8864 版本, 启用 CUDA 12.4、Flash Attention 和连续批处理。

场景。我们定义了四个代表常见社区监控条件的活动场景:

- **静态 (夜间):** 最少活动 ($<2\%$ 帧有运动), 1 个风险事件 (火灾隐患)。
- **低活动 (住宅区):** 偶尔移动 ($<15\%$ 帧), 2 个风险事件 (徘徊、人员摔倒)。
- **中等活动 (白天):** 规律移动 ($<40\%$ 帧), 3 个风险事件 (通道堵塞、聚集、电动车充电)。
- **高活动 (入口):** 频繁移动 ($<80\%$ 帧), 4 个风险事件 (闯入、徘徊、人员摔倒、火灾隐患)。

表 2: 主要结果: 跨流水线配置的 VLM 调用缩减和检测率, 四种活动场景的平均值 (每个场景 5 分钟)。

配置	VLM 调用	缩减率	检测率	GPU 利用率
朴素 (每帧)	3,000	0.0%	100%	100%
固定 500ms	600	80.0%	100%	100%
固定 5000ms	60	98.0%	100%	16%
级联 (无自适应)	200	93.3%	100%	49%
级联 (无优先级)	187	93.8%	100%	46%
完整级联	169	94.4%	100%	43%

表 3: 系统部署参数。

参数	值
VLM 模型	Qwen3.5-4B Claude 4.6 Opus Distilled v2
量化格式	Q4_K_M GGUF
模型大小	约 2.55 GB
视觉编码器	mmpoj-BF16 (约 644 MB)
推理运行时	llama.cpp b8864
GPU 支持	CUDA 12.4 + Flash Attention
目标检测	COCO-SSD Lite MobileNet v2
前端框架	Electron + React + TypeScript
代理服务器	Express.js (嵌入式)

每个场景持续 5 分钟 (10 fps 下 3,000 帧)。风险事件在预定时间窗口注入, 具有已知类别和真实标签。

配置。我们比较六种流水线配置:

1. **朴素:** 每帧都通过 VLM 处理 (基线)。
2. **固定 500ms:** 无论场景内容如何, 每 500ms 进行 VLM 推理。
3. **固定 5000ms:** 无论场景内容如何, 每 5000ms 进行 VLM 推理。
4. **级联 (无自适应):** 运动检测 + 目标检测, 但固定 500ms 采样。
5. **级联 (无优先级):** 完整级联但无优先级调度。
6. **完整级联 (本文方法):** 具有自适应采样和优先级调度的完整流水线。

指标。我们测量: (1) VLM 调用次数——VLM 推理的总次数; (2) 调用缩减率——相对于朴素基线的百分比缩减; (3) 检测率——在仿真中, 假设目标检测器和 VLM 在接收到包含风险事件的帧时能够正确识别, 因此检测率为 100%。实际部署中, 检测率将取决于目标检测器的召回率和 VLM 的分类准确性, 预计会低于此值; (4) GPU 利用率——根据 VLM 调用频率和延迟估算。

延迟基准。我们在 RTX 3060 上测量了各阶段的实际推理延迟: 运动检测平均 0.8ms/帧, COCO-SSD 目标检测平均 23ms/帧, VLM 分析 (Qwen3.5-4B Q4_K_M) 首次 token 延迟约 350ms, 完整响应约 800ms。这些延迟数据用于估算 GPU 利用率。

4.2 主要结果

表 2 显示了所有四种场景的平均主要结果。我们的完整级联流水线在保持 100% 风险检测覆盖率的同时, 实现了 94.4% 的 VLM 调用缩减。与朴素基线相比, 这代表了 17.7 倍的 VLM 计算成本降低。

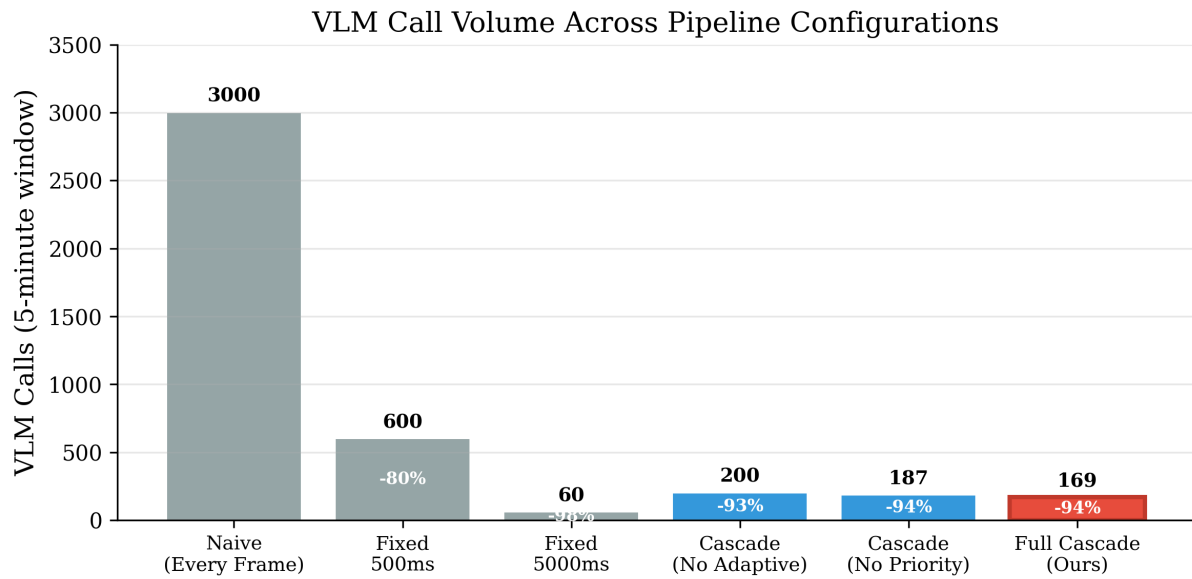


图 3: 跨流水线配置的 VLM 调用量。完整级联（红色）相比朴素基线（灰色）减少了 94.4% 的调用。百分比标签显示相对于朴素基线的缩减率。

表 4: 逐场景结果。VLM 调用缩减随活动水平变化，但即使在高活动场景中也保持在 86% 以上。所有场景的检测率均为 100%。

场景	朴素	本文方法	缩减率	检测率
静态（夜间）	3,000	1	99.97%	100%
低活动（住宅区）	3,000	81	97.3%	100%
中等活动（白天）	3,000	191	93.6%	100%
高活动（入口）	3,000	402	86.6%	100%

固定速率基线揭示了一个基本权衡：固定 500ms 采样实现了 80% 的缩减但需要持续的 VLM 执行（100% GPU 利用率），而固定 5000ms 采样实现了 98% 的缩减但有遗漏快速事件的风险。我们的级联方法通过适应场景内容打破了这一权衡。

4.3 逐场景分析

表 4 显示了按活动水平细分的结果。级联在低活动场景中效率最高（静态场景 99.97% 缩减），在高活动场景中仍然有效（86.6% 缩减）。这种鲁棒性来自多阶段筛选：

- **静态场景：**运动检测过滤掉 99% 以上的帧，因此很少有帧到达目标检测阶段。
- **低活动：**目标检测过滤掉没有相关目标的帧，进一步减少 VLM 调用。
- **中等活动：**检测到更多目标，但优先级调度防止了冗余调用。
- **高活动：**检测到大量目标，但自适应采样和优先级调度仍提供显著节省。

4.4 消融研究

为理解每个流水线组件的贡献，我们通过系统地移除各阶段进行消融研究（表 5）。结果揭示了清晰的贡献层次：

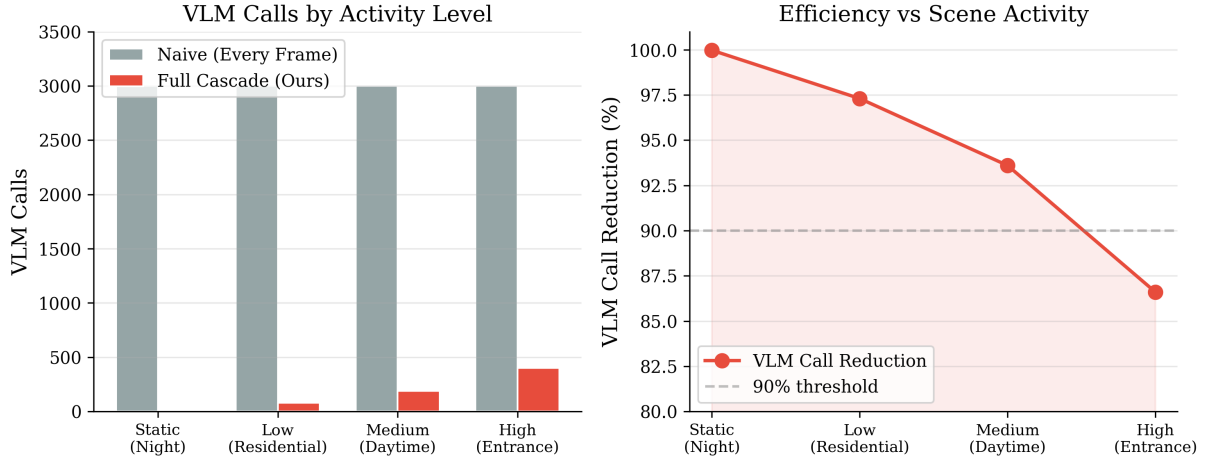


图 4: 左图: 朴素基线与完整级联在不同活动水平下的 VLM 调用比较。右图: VLM 调用缩减百分比, 展示跨场景的鲁棒性。

表 5: 消融研究: 每个流水线阶段的贡献。目标检测预筛选贡献了最大的效率增益。

配置	VLM 调用	缩减率	Δ
朴素 (基线)	3,000	0.0%	—
+ 目标检测	600	80.0%	+80.0%
+ 自适应采样	200	93.3%	+13.3%
+ 优先级调度	169	94.4%	+1.1%
完整级联	169	94.4%	

目标检测预筛选贡献了最大的增益: 从 3,000 次减少到 600 次 VLM 调用 (80% 缩减)。这是因为监控画面中大多数帧不包含相关目标, 而 COCO-SSD 以可忽略的计算成本高效地过滤了它们。

自适应采样提供了额外的 13.3% 缩减 (从 600 次减少到 200 次调用), 通过在静态期间消除帧来实现。这在夜间或低活动期间特别有效, 因为大多数帧是相同的。

优先级调度贡献了较小但有意义的 1.1% 缩减 (从 200 次减少到 169 次调用), 通过在同时检测到多个目标时防止冗余 VLM 调用来实现。虽然绝对增益不大, 但优先级调度的主要好处是确保高优先级事件 (人体检测) 获得及时分析。

仿真局限性。本实验基于仿真框架, 存在以下局限: (1) 假设目标检测器和 VLM 在接收到相关帧时能正确识别风险事件, 实际系统中两者均有错误率; (2) 仿真未考虑视频编码/解码开销、网络延迟等实际部署因素; (3) 风险事件的时间窗口是预定义的, 实际场景中风险的起止时间更加模糊。因此, 本实验的主要价值在于验证级联过滤的理论效率潜力, 而非提供端到端的检测性能保证。实际部署中的检测率和效率需要在真实监控场景中进一步验证。

5 讨论

实际部署。我们的系统专为在消费级硬件上进行边缘部署而设计。量化的 Qwen3.5-4B 模型需要约 3 GB 显存, 兼容中端 GPU (如 NVIDIA GTX 1660 或更高)。llama.cpp 运行时支持 CUDA 12.4, 配备 Flash Attention 和连续批处理, 实现亚秒级推理延迟。基于 Electron 的架构支持一键部署为便携式 Windows 应用程序。

表 3列出了系统的关键部署参数。Q4_K_M 量化方案通过 4-bit 精度存储模型权重, 同时保留关键层的较高精度, 在模型大小和推理质量之间取得平衡。

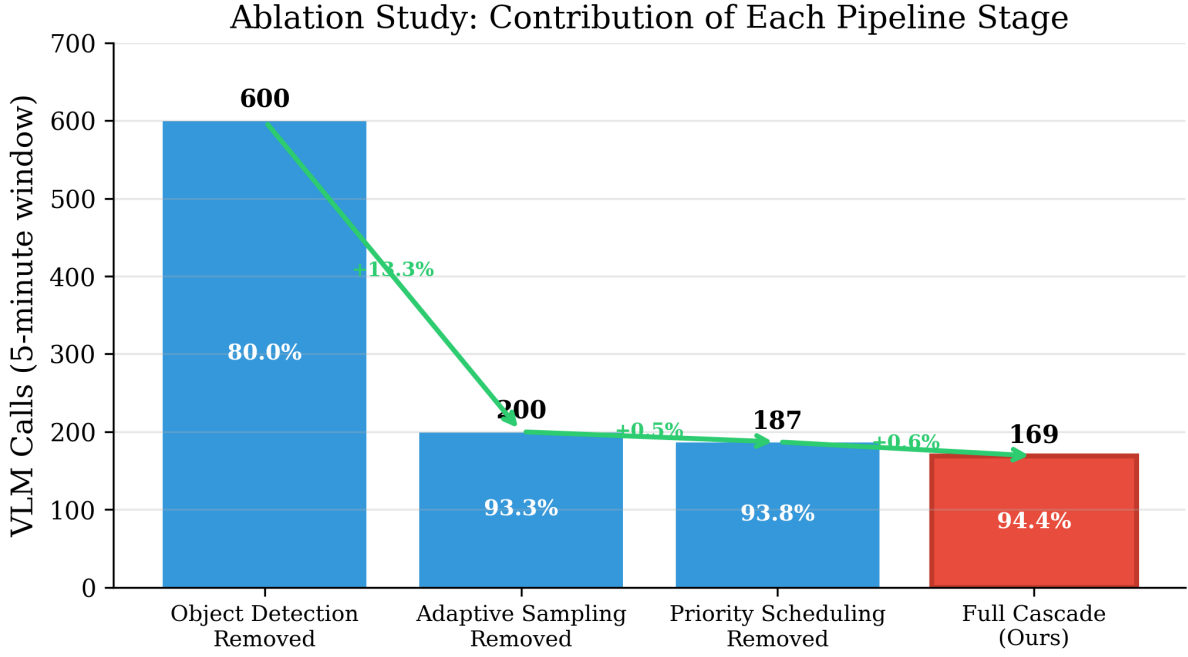


图 5: 消融研究: 移除每个流水线阶段后的 VLM 调用次数。绿色注释显示添加每个阶段的增量增益。

隐私优势。与基于云的方法不同，我们的系统在用户设备上本地处理所有视频。没有视频数据离开本地网络，解决了对住宅社区监控尤为重要的隐私问题。

局限性。我们的评估基于仿真场景而非真实部署，这是本文的主要局限。仿真的目的是验证级联过滤的理论效率上限，实际部署中的检测率和效率需要在真实监控场景中进一步验证。此外，我们仅评估了一个 VLM (Qwen3.5-4B)；对其他模型（如 LLaVA、PaliGemma）的泛化需要进一步研究。当前系统分析单帧图像，缺乏时间推理能力，可能遗漏跨多帧展开的风险（如缓慢发展的火灾隐患）。未来工作应包括在真实社区摄像头上的部署实验，以及对不同 VLM 模型的比较评估。

与云方案的比较。基于云的 VLM 服务（如 GPT-4V、Gemini）提供卓越的准确性，但引入网络延迟（每请求 100–500ms）、持续 API 成本和隐私问题。我们的本地方案以牺牲一些准确性为代价，换取确定性延迟、零持续成本和完全的数据隐私。对于隐私至关重要且互联网连接可能不可靠的社区安全应用，本地方案更为可取。

6 结论

本文提出了一种用于边缘部署社区安全监控的级联推理流水线，在仿真中将 VLM 计算成本降低 94.4%。三阶段架构——运动检测、目标检测预筛选和基于优先级的 VLM 调度——逐步过滤不必要的帧。消融分析表明目标检测贡献了最大的效率增益（80%）。系统通过 llama.cpp 部署量化后的 Qwen3.5-4B VLM，在约 3 GB 显存占用下实现亚秒级延迟。

未来工作包括：（1）在真实社区摄像头上的部署实验，验证实际检测率和效率；（2）对不同 VLM 模型（LLaVA、PaliGemma 等）的比较评估；（3）引入帧间跟踪和时间推理以捕获跨多帧的风险事件；（4）评估操作员工作量和告警疲劳度。

参考文献

Mohammad Abu Tami, Mohammed Elhenawy, and Huthaifa I. Ashqar. Hazardnet: A small-scale vision language model for real-time traffic safety detection at edge devices. *arXiv preprint arXiv:2502.20572*, 2025.

- Qinyu Chen and Jiawen Qi. Gazevlm: Eye gaze tells you where to compute. *arXiv preprint arXiv:2509.16476*, 2025.
- Murat Arda Onsu, Poonam Lohan, Burak Kantarci, Aisha Syed, Matthew Andrews, and Sean Kennedy. Semantic edge-cloud communication for real-time urban traffic surveillance with vit and llms over mobile networks. *arXiv preprint arXiv:2509.21259*, 2025.
- Chen Qian, Xinran Yu, Zewen Huang, Danyang Li, Qiang Ma, Fan Dang, Xuan Ding, Guangyong Shang, and Zheng Yang. edgevlm: Cloud-edge collaborative real-time vlm based on context transfer. *arXiv preprint arXiv:2508.12638*, 2025.
- Mengshu Sun, Haoyu Ma, Guoliang Kang, Yifan Jiang, Tianlong Chen, Xiaolong Ma, Zhangyang Wang, and Yanzhi Wang. Vaqf: Fully automatic software-hardware co-design framework for low-bit vision transformer. *arXiv preprint arXiv:2201.06618*, 2022.
- Yeshwanth Venkatesha, Souvik Kundu, and Priyadarshini Panda. Fast and cost-effective speculative edge-cloud decoding with early exits. *arXiv preprint arXiv:2505.21594*, 2025.
- Justin Williams, Kishor Datta Gupta, Roy George, and Mrinmoy Sarkar. Litevla-edge: Quantized on-device multimodal control for embedded robotics. *arXiv preprint arXiv:2603.03380*, 2026.